webRagSys — 社内文書検索・AIチャットシステム

webRagSysは、社内文書を登録・管理し、質問に関連する文書を検索して、その内容を根拠にAIが回答するWeb APIシステムです。FastAPIとLangChainによるRAGを実装し、Fake・OpenAI・ローカルOllama（Qwen3.5）を切り替えながら、文書検索から回答生成までの流れを学べます。

システム概要

- 文書管理（CRUD）とAIチャット（RAG）を提供するFastAPIシステム
- 処理の本質：Python → API → DB → LLM → RAG
- 登録文書をテキスト分割→Embedding→Vector Databaseへ保存し、質問時に類似検索してLLMへ渡す

主な機能

- 文書登録、一覧取得、詳細取得、更新、削除
- AIチャット：質問→Embedding→類似文書検索→LLM回答
- RAG：LangChain利用（TextSplitter、Embeddings、VectorStore、Retriever、Chain）
- 既定は課金回避のFake実装。OpenAI APIまたはローカルOllama（Qwen3.5）へ切替可能

使用技術

項目	内容
言語	Python 3.10+（実測3.11）
API	FastAPI、Pydantic
DB	SQLite（ローカル動作確認済み）、PostgreSQL＋pgvector（拡張の導入が必要）、SQLAlchemy
RAG	LangChain、langchain-ollama
LLM	Fake（既定）、OpenAI API、ローカルOllama（qwen3.5:4b）
Embedding	Fake（8次元）、OpenAI、ローカルOllama（qwen3-embedding:0.6b、1024次元）
テスト	pytest
Lint	ruff
環境	Git／GitHub、venv（Docker／docker-composeは構成見本のみで起動対象外）

システム構成

```
webRagSys/
├── AGENTS.md
├── README.md
├── config.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .env.example
├── src/api/        → main.py、routers/documents.py、routers/chat.py
├── src/schemas/    → Pydanticスキーマ
├── src/db/         → database.py、models.py
├── src/rag/        → chunker.py、embeddings.py、vectorstore.py、chain.py
├── src/llm/        → provider.py、fake_provider.py
├── src/utils/
├── tests/
```

```
├── docs/          → beginner_overview.md、handover.md
└── diagrams/      → Mermaid図（.mmd＋.md＋.pdf）
```

セットアップ方法

```
# 0. プロジェクト直下へ移動
cd <プロジェクト直下>

# 1. 仮想環境作成（初回のみ）
python -m venv .venv

## PowerShellで有効化
.¥.venv¥Scripts¥Activate.ps1

## コマンドプロンプト（cmd）で有効化
.¥.venv¥Scripts¥activate.bat

# 2. 依存導入（有効化したPowerShellまたはcmdで実行）
pip install -r requirements.txt

# 3. 環境変数ファイル作成（公開用見本から複製）
copy .env.example .env

# 4. .env編集（偽実装のままならキー不要）
# USE_FAKE=true
# 本番のみ以下を設定
# OPENAI_API_KEY=xxxx
# USE_FAKE=false
# ローカル起動はSQLite指定（Docker対象外のため）
# DATABASE_URL=sqlite:///./app.db
```

起動方法

PowerShellで仮想環境を有効化

```
.¥.venv¥Scripts¥Activate.ps1
```

コマンドプロンプト（cmd）で仮想環境を有効化

```
.¥.venv¥Scripts¥activate.bat
```

有効化後、プロンプトに（.venv）が表示されていることを確認します。どちらのシェルでも、次のコマンドでAPIを起動できます。

```
uvicorn src.api.main:app --reload
```

Docker起動は対象外とする（storage制約のため、2026-09-19決定）。Dockerfileとdocker-compose.ymlは構成見本として残す。

動作確認

- http://127.0.0.1:8000/health → `{"status":"ok"}` 表示で生存確定
- http://127.0.0.1:8000/docs → 操作画面表示
- http://127.0.0.1:8000/documents → 初期は `[]` 表示で正常
- `/` は未定義のため404が正常

APIー覧

メソッド	エンドポイント	内容
GET	/documents	文書一覧取得
GET	/documents/{id}	文書詳細取得
POST	/documents	文書登録
PUT	/documents/{id}	文書更新
DELETE	/documents/{id}	文書削除
POST	/chat	AIチャット

リクエスト／レスポンスはJSONを使用する。

RAGの処理概要

```
文書
 ↓
テキスト分割（LangChain TextSplitter）
 ↓
Embedding（本番：外部API／既定：FakeEmbeddings）
 ↓
Vector Databaseへ保存（本番pgvector／SQLite時はJSON列で代替）

ユーザー質問
 ↓
Embedding
 ↓
類似文書検索（pgvectorまたはPython側コサイン計算＋LangChain Retriever）
 ↓
検索結果をコンテキスト化
 ↓
LLM（本番：外部API／既定：Fake LLM）
 ↓
回答（＋参照文書ID）
```

使用方法

```
# 文書登録例
curl -X POST http://localhost:8000/documents ^
  -H "Content-Type: application/json" ^
  -d "{¥"title¥": ¥"休暇規定¥", ¥"content¥": ¥"年次休暇は...¥"}"

# 質問例
curl -X POST http://localhost:8000/chat ^
  -H "Content-Type: application/json" ^
  -d "{¥"question¥": ¥"休暇申請の手順は？¥"}"
```

テスト方法

```
# 全テスト（Fakeは外部通信なし。Ollamaケースはローカルollamaを使用）
python -m pytest tests -q

# Lint
python -m ruff check src tests config.py
```

テスト内容：

- 文書：登録、取得、更新、削除、存在しないIDのエラー
- チャット：質問送信、偽LLM回答、登録文書利用、APIエラー時処理
- backend組合せ：fake／Ollama × SQLite／PostgreSQL。PostgreSQLケースは `TEST_DATABASE_URL` とpgvectorが必要で、利用不可時はskip

ドキュメント

- `docs/beginner_overview.md` ：Python中級・RAG初心者向け解説（実コード照合済み、PDF付き、5章に操作と内部の対応表あり）
- `docs/beginner_overview_RevA.md` ：上記の別名保存版（です調統一、略語展開、SQLite正規明記、PDF付き）
- `docs/beginner_overview_RevF.md` ：Rev.E継承版（Embeddingの出力形式、文書・質問のベクトル化、次元数の傾向を補足、PDF付き）
- `docs/user_manual.md` ：取扱説明書（起動から/docs操作、登録、会話、異常時まで、PDF付き）
- `py_source_export/webRagSys_source.pdf` ：全ソース結合PDF（行番号・ハイライト付き、config＋src＋testsの26件）
- `diagrams/*_RevA.md` ：図6件の別名保存版（読み方と実コード対応付き、.mmd＋.pdf）
- `docs/handover.md` ：進捗管理（方針→実装→結果→残タスク→実行コマンド、PDF付き）
- `diagrams/` ：フローチャート、シーケンス、クラス（RAG／モジュール）、mindmap、状態遷移（.mmd＋.md＋.pdf）
- 図PDF生成： `diagrams/convert_to_pdf.ps1` （mermaid-cli、前提Node.js）
- docs PDF生成：md-to-pdf＋ `docs/pdf_config.json` （例： `npx md-to-pdf docs/beginner_overview_RevF.md --config-file docs/pdf_config.json` 、ページ番号付き）

今後の拡張案

- ユーザー認証、文書ファイルアップロード、PDF／Word対応
- 文書カテゴリ管理、会話履歴、回答の出典表示
- Webフロントエンド、クラウドデプロイ、CI/CD
- 高度なRAG検索、AIエージェント機能

完成条件（初期版）

1. FastAPI起動、文書CRUD動作（ローカル起動で確認、DBはPostgreSQLまたはSQLite退避）
2. Fake（既定）、OpenAI APIまたはローカルOllamaを用いた回答生成
3. Embedding化、Vector保存、類似検索、RAG回答
4. GitHubでREADME確認、基本テスト実行
5. Docker構築は対象外（storage制約のため、2026-09-19決定）

注意点

- `.env` はGit管理対象外。APIキー等を公開しないこと
- docsとdiagramsはテストの後に作成し、PDF生成すること